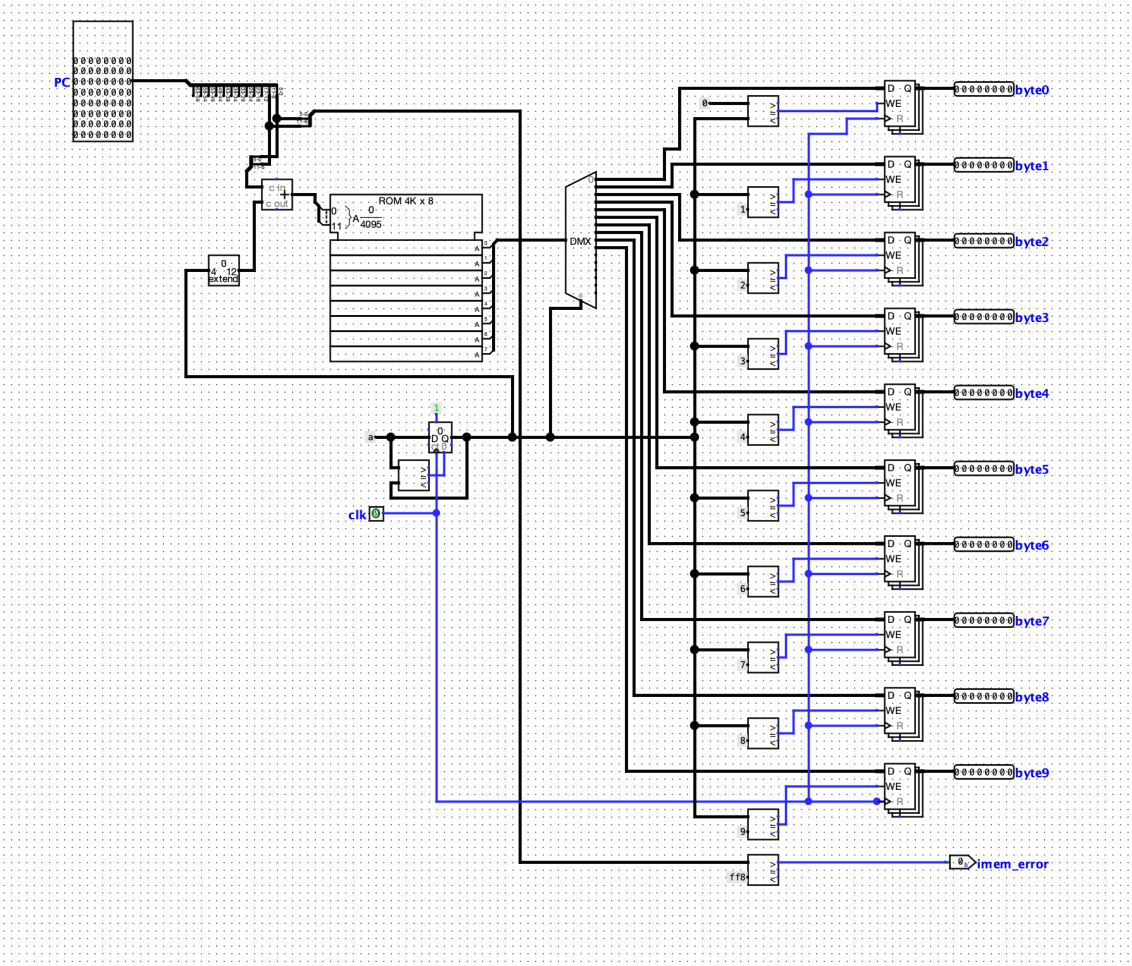
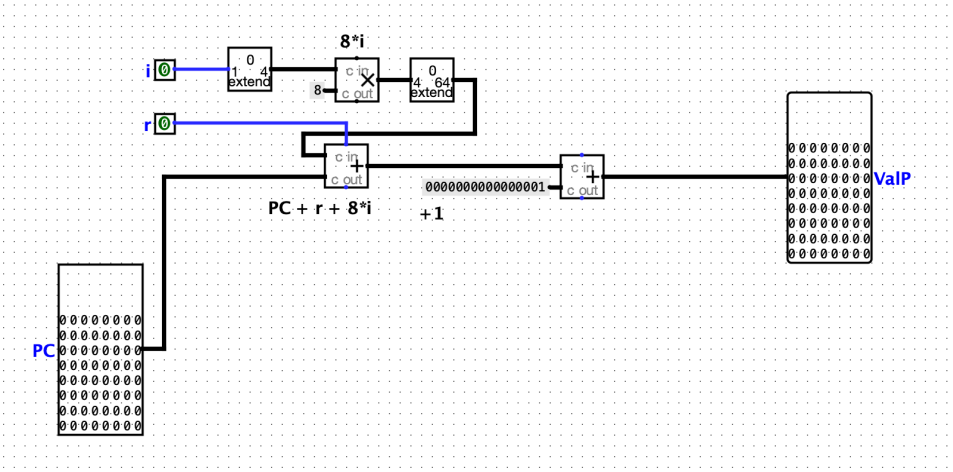
Group 27

Lab 6 Report

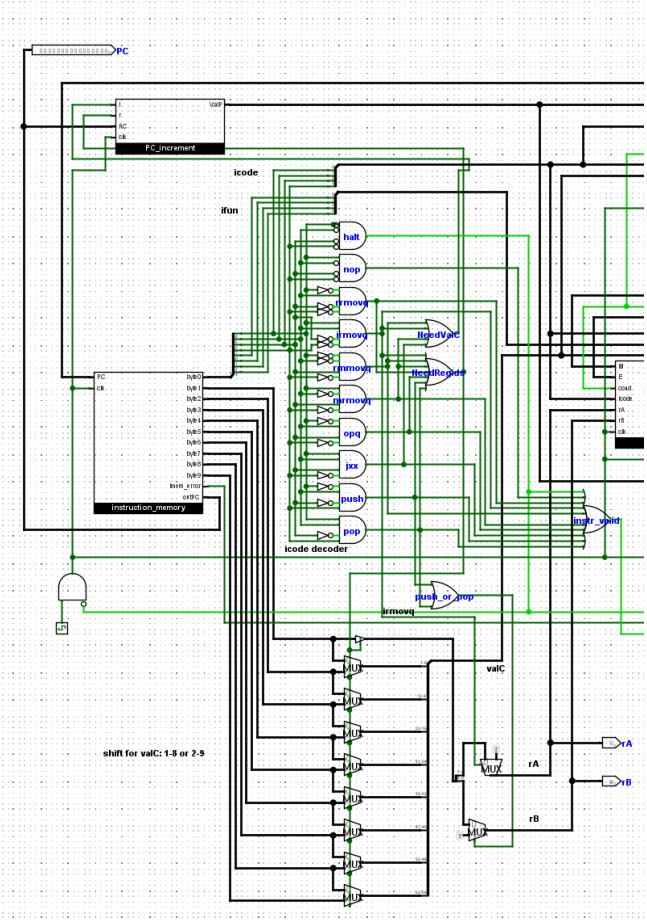
Fetch:



The instructions originate within the ROM, preloaded before runtime. Fetch will read from ROM at the current address within the value PC. The PC value will start at address 0, and increment per instruction. Each clock cycle will read one byte of instruction from ROM and store it in a register, until a counter counts 10 clock cycles. imem_error represents an error within the instruction memory unit that results from accessing a memory that is larger than the 12 bits that are needed.



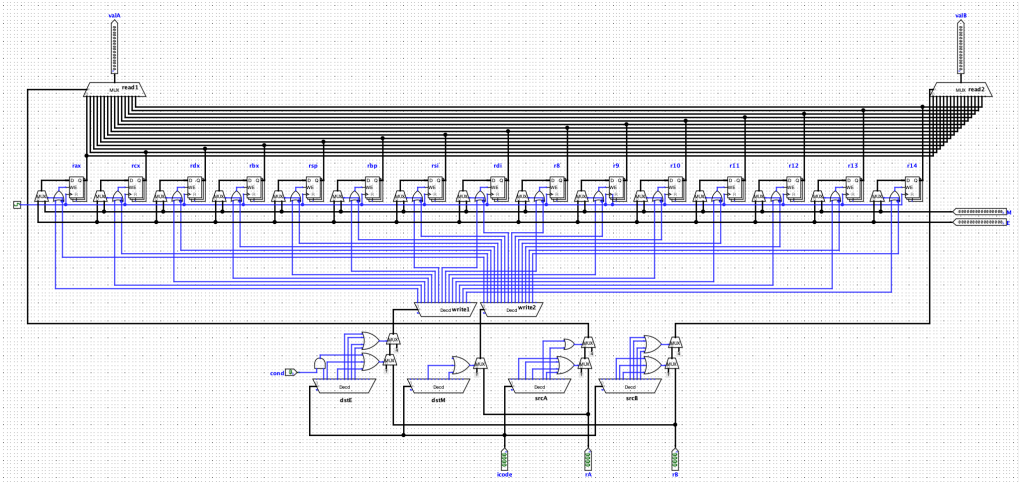
This image shows the contents of PC_increment, which takes in icode, and values i and r. i and r represent NeedValC and NeedRegIDs. With these values, it calculates the next ValP address value from the formula: PC + 1 + 8i + r



This image demonstrates the entire circuit. From instruction_memory, 10 bytes are extracted. The first one represents icode:ifun, which are determined by a series of and gates making a

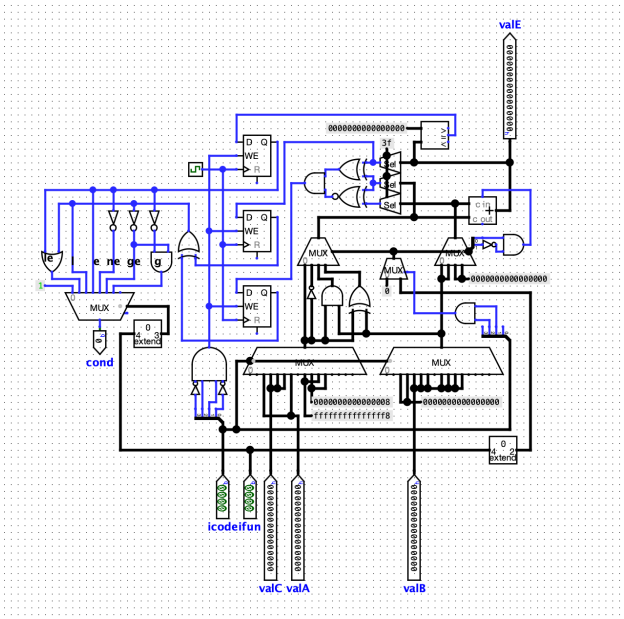
multiplexer. The second one corresponds to registers, if needed. The remaining are placed into valC, which requires either bytes 1-8 or 2-9. Flags for NeedRegIDs and NeedValC are created from the mux AND gates. If the icode instruction is one of the expected Y86 instructions, the instr_valid flag is set.

Decode:



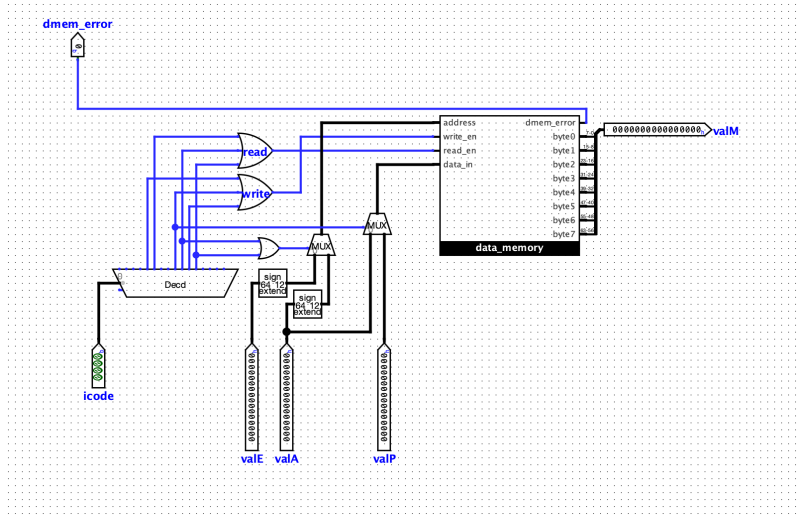
Decode incorporates several muxes, with icode as the 4 selector bits, that determine the registers needed to store data in. The inputs valA and valB are used to select the register. On the right hand side, inputs for valE and valM feed into the registers as well after the writeback stage completes.

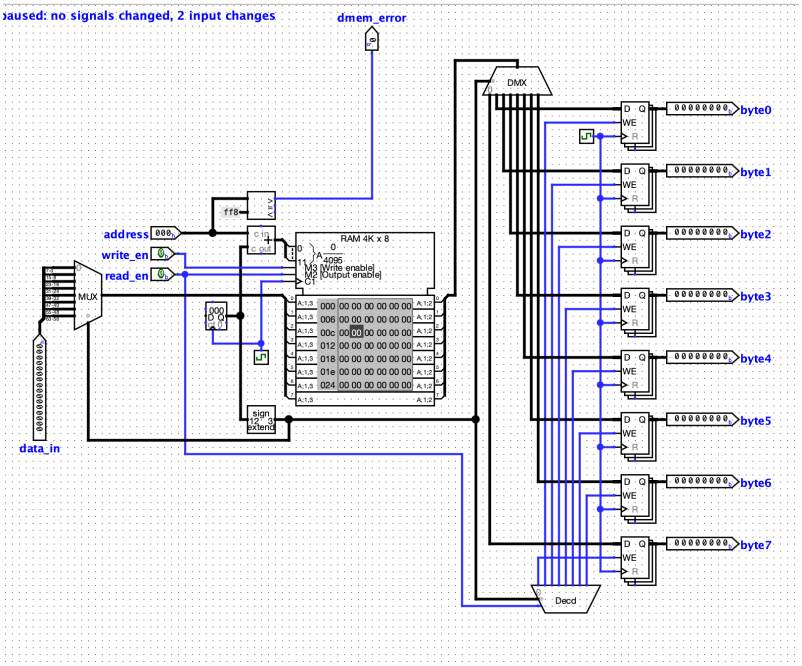
Execute:



The Execute stage consists of 2 central muxes, again with icode as the selector bits. The inputs to the left mux are valC, valA, -8, and 8. The inputs to the right mux are valB and zero. Once the two operands are selected, icode and ifun select the operator with a mux. The two inputs are passed through the ALU extenders and are added together. The result of the ALU is output as valE and also passed to condition code. In CC, overflow is checked by passing the operands through combinational logic and storing them into the 3rd register. Signed is checked by getting the most significant bit of valE and stored in the 2nd register. And Zero is checked by using a comparator and stored in the 1st register. More combination logic and a mux controlled by ifun determines the flag ‘cond’.

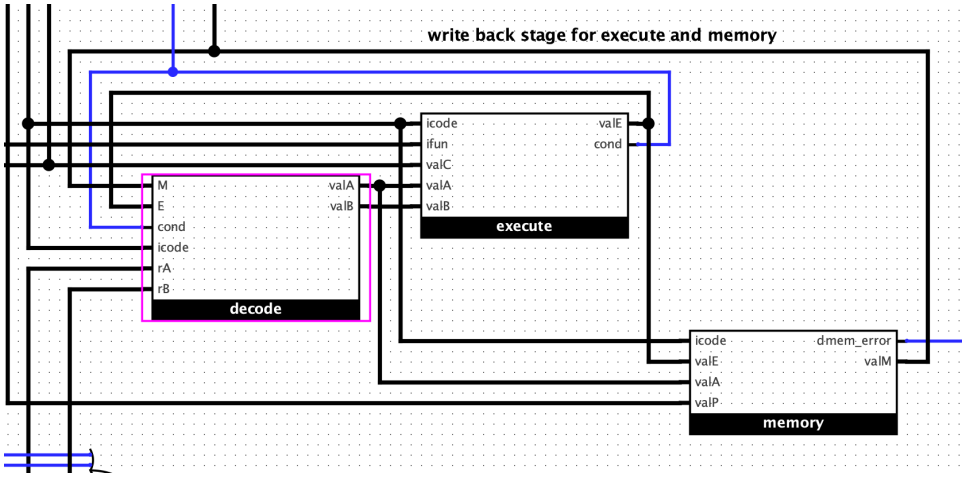
Memory:





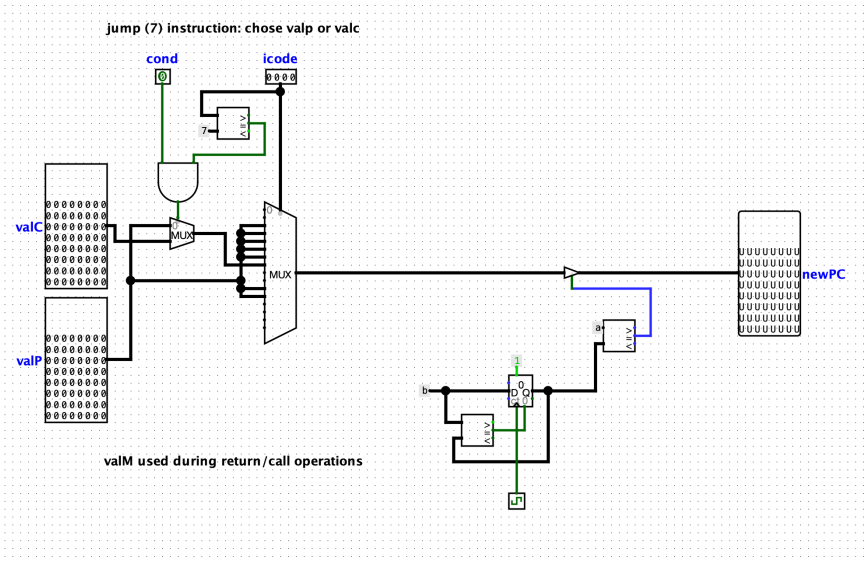
This memory works similar to the instruction memory from fetch; however, we implemented a RAM unit because we are able to write information into the RAM with the given address that is passed through the instructions. There are also read and write enablers that will toggle based on what the instruction needs, which is determined in the icode decoder in the first image. The registers then output 8 bytes of data comprising valM, which is read from the ROM and will be sent back to the registers during the write back stage.

Write Back:



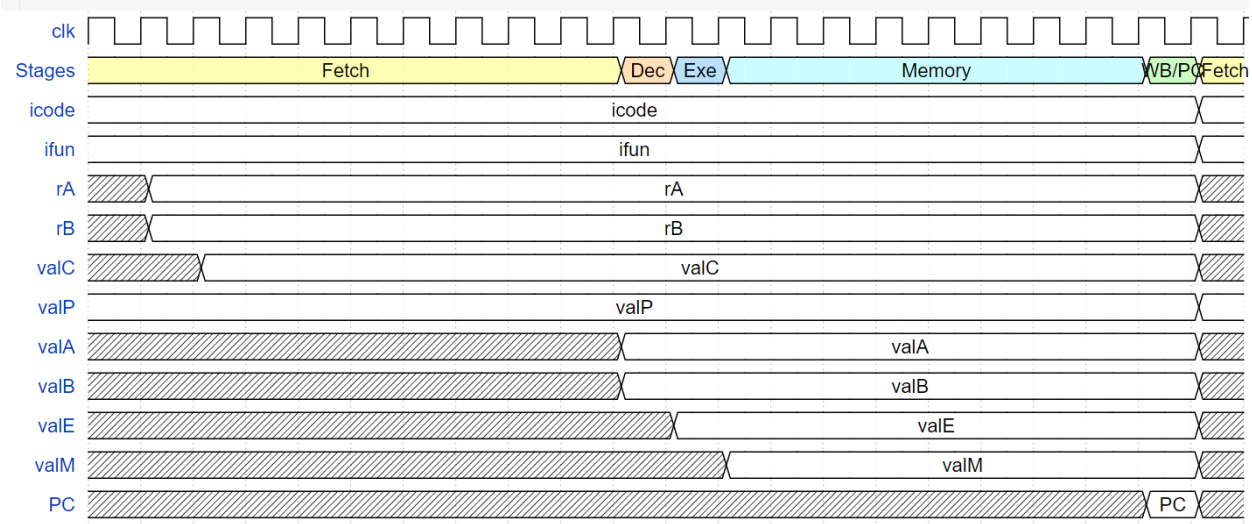
The write back stage takes the calculated valE value from the execute unit and valM from the memory unit and loads it to the registers in the decode unit.. In our implementation, we have write signals that will turn on when these instructions are reached, and they will take in the register values and place it in the correct registers.

PC Update:



PC update takes in valC and valP, where valP is calculated using the PC increment given the current PC plus the conditions that were given in fetch. The mux here chooses based on whether or not icode contains the value 7, meaning the jmp instruction will use valC to update valP. If icode is 7, and jump instruction is true, the PC will jump to valC, otherwise the PC will jump to the next instruction calculated from the PC_update circuit.

Timing Diagram:



The CPU runs on a 20 clock edge cycle. Fetch executes first, retrieving icode and ifun from the first byte of data read from the instruction memory on the next clock cycle. The first instruction is fetched from the address provided by valP. The next byte of data contains rA and rB, which is obtained on the next clock cycle. The next 8 clock cycles are used to retrieve valC. Fetch takes a total of 10 clock cycles. The decode stage needs a single clock cycle to use rA, rB, valE, and valM to update its registers and produce valA and valB. Execute also needs a clock cycle to produce valE in a register from valA, valB, and valC. Memory needs 8 clock cycles to read from the RAM, 8 bytes at a time, within the data memory. It takes one clock cycle for the value from the RAM, valM, and valE from execute to be written back to the decode registers. Using this same clock cycle, the value for the address of the next instruction is calculated, hence the PC update stage. Fetch is started again on the next clock edge, meaning the total process takes 21 clock cycles.

Assembly Instructions

We wrote 3 assembly programs for testing the functionality of our CPU

1) Matrix Addition
    a) Instructions halt, irmovq, rrmovq, rmmovq, and mrmovq are tested in this program
2) Bubble sort
    a) This program tests: irmovq, rmmovq, mrmovq, opq, jmp, pushq, popq
3) Addition factorial
    a) irmovq, rmmovq, opq, jmp, and pushq are all included in this program